aiida-n2p2: workflow and restart API

Irina Piazza

November 4, 2026

Contents

1 n2p2 workflow **2**
- 1.1 Workflow architecture . . . 2
- 1.2 The workflow input API . . . 3
  - 1.2.1 What is an input API? . . . 3
  - 1.2.2 Input API of `MakeNNPWorkchain` (v0.2.0) . . . 3
- 1.3 CalcJobs and parsers . . . 5
- 1.4 Structured inputs and restart (v0.3.0) . . . 5
  - 1.4.1 Data types . . . 5
  - 1.4.2 Two restart modes compared . . . 6
  - 1.4.3 Manual restart API . . . 7
  - 1.4.4 Automatic restart API (v0.4.0) . . . 8
- 1.5 Memory-conscious retrieval defaults . . . 8
- 1.6 Regression tests . . . 9
- 1.7 Updated examples . . . 9

1 n2p2 workflow

The `aiida-n2p2` plugin (version 0.4.0) provides an AiiDA-native interface to the n2p2 neural-network potential package. The workflow layer follows AiiDA best practices: each pipeline stage is exposed as an independent `WorkChain`, and a composite work chain orchestrates the full potential-building procedure. Version 0.4.0 adds automatic restart when a training CalcJob stops before reaching the target number of epochs (e.g. after scheduler walltime limits), while continuing to merge learning curves across segments. This section summarises the architecture, the workflow *input API*, memory-conscious retrieval defaults, restart behaviour, and the regression test suite.

1.1 Workflow architecture

The end-to-end pipeline consists of three physical steps:

1. **Scaling** (`nnp-scaling`): generation of symmetry-function scaling data from the training set;
2. **Training** (`nnp-train`): optimisation of neural-network weights using the scaled descriptors;
3. **Validation** (optional, via `aiida-lammps`): a short LAMMPS molecular-dynamics run to verify the trained potential.

Each step is wrapped in a dedicated work chain and can be submitted independently or composed through `MakeNNPWorkchain`. Figure 1 shows the vertical call order (single-column layout).

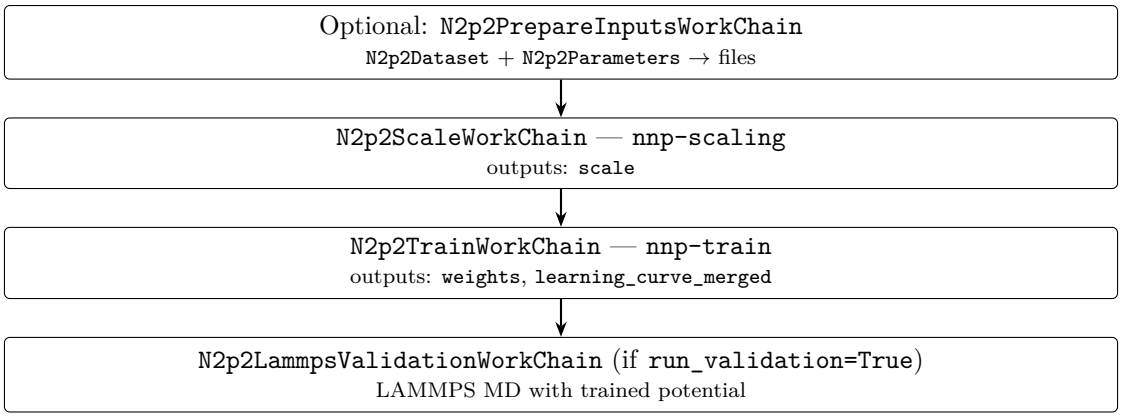


Figure 1: `MakeNNPWorkchain` pipeline in top-down order. The prepare step runs when structured inputs or overrides are supplied; scaling forwards `inputData`, `inputNN`, and `scale` to training; validation is skipped when `run_validation=False`.

Table 1: AiiDA workflow entry points registered by `aiida-n2p2` v0.4.0.

Entry point	Class	Purpose
`n2p2.scale`	`N2p2ScaleWorkChain`	Run scaling
`n2p2.train`	`N2p2TrainWorkChain`	Run training (with restart)
`n2p2.validate_lammps`	`N2p2LammpsValidationWorkChain`	LAMMPS validation
`n2p2.prepare_inputs`	`N2p2PrepareInputsWorkChain`	Build `input.data`/`input.nn`
`n2p2.make_potential`	`MakeNNPWorkchain`	Full pipeline

`MakeNNPWorkchain` exposes inputs from the three sub-work chains under the namespaces `scaling`, `training`, and `validation`. Scaling outputs (`inputData`, `inputNN`, `scale`) are forwarded automatically to the training step; trained weights and network files are forwarded

to the validation step when enabled. Workflow outputs are `potential` (best weights file), `scale` (scaling data), `last_weights` (checkpoint for restart), `learning_curve_merged`, and `learning_curve_plot_data`.

Validation is controlled by the top-level boolean input `run_validation` (default: `True`). When set to `False`, the work chain stops after training.

1.2 The workflow input API

1.2.1 What is an input API?

In software engineering, an **API** (*Application Programming Interface*) is the set of rules that defines *how* a user or another program interacts with a piece of software: which functions can be called, which arguments they accept, and what they return. For example, a web service might expose an API with endpoints such as `GET /users/{id}`; clients must follow that contract to obtain data.

In AiiDA, a **workflow input API** plays an analogous role. Each `WorkChain` declares a fixed list of **input ports**—named slots that accept AiiDA data nodes (e.g. `Code`, `SinglefileData`, `Int`, `Dict`). Together, these ports form the *input schema* of the workflow: they specify

- **which** quantities must be provided (e.g. training data, executable, atomic number);
- **under which names** they must be passed (e.g. `inputData` rather than `data_file`);
- **which AiiDA type** each input must have (e.g. `Int` for the atomic number);
- whether an input is **required or optional**, and its **default** value if optional;
- how inputs are **grouped** into **namespaces** when a workflow composes several sub-workflows (e.g. `scaling.code`, `training.code`).

From the user's perspective, launching a workflow means building a Python dictionary (`inputs`) whose keys and nested structure must match this schema exactly. AiiDA validates the dictionary before submission; if a required port is missing or has the wrong type, the submission fails immediately. This validation is what makes the input API a stable *contract* between the plugin developer and the end user.

When a plugin is updated, the input API may change: ports can be renamed, regrouped, or moved between namespaces. Such changes are called **breaking changes** if existing scripts must be rewritten to work with the new version. Documenting the input API is therefore essential for reproducibility and for migrating calculation scripts across plugin releases.

1.2.2 Input API of `MakeNNPWorkchain` (v0.2.0)

Version 0.2.0 refactors the input API of `MakeNNPWorkchain`. The previous single nested namespace `n2p2` was replaced by three top-level namespaces—`scaling`, `training`, and `validation`—plus the boolean flag `run_validation`. Port names in the validation step were also aligned with AiiDA conventions (`script`, `structure`).

Previous input API (pre-0.2.0).

```
inputs = {
    "n2p2": {
        "scale": {
            "code": scale_code,
            "nbin": nbin,
            "inputData": input_data,
            "inputNN": input_nn,
            "metadata": scale_metadata,
```

```
        },
        "train": {
            "code": train_code,
            "atomicNumber": atomic_number,
            "metadata": train_metadata,
        },
        "validate": {
            "code": lammps_code,
            "lammpsScript": lammps_script,
            "lammpsData": lammps_structure,
            "metadata": validate_metadata,
        },
    }
}
```

Current input API (v0.2.0).

```
from aiida.orm import Bool

inputs = {
    "scaling": {
        "code": scale_code,
        "nbin": nbin,
        "inputData": input_data,
        "inputNN": input_nn,
        "metadata": scale_metadata,
    },
    "training": {
        "code": train_code,
        "atomicNumber": atomic_number,  # e.g. Int(13) for Al
        "metadata": train_metadata,
    },
    "validation": {
        "code": lammps_code,
        "script": lammps_script,        # was: lammpsScript
        "structure": lammps_structure,  # was: lammpsData
        "metadata": validate_metadata,
        "retrieve_trajectory": Bool(False),
    },
    "run_validation": Bool(True),
}
```

Table 2 summarises the main differences between the two versions.

Table 2: Summary of input API changes in `aiida-n2p2` v0.2.0.

Aspect	Pre-0.2.0	v0.2.0
Top-level namespace	`n2p2`	`scaling`, `training`, `validation`
Scaling sub-namespace	`scale`	`scaling`
Training sub-namespace	`train`	`training`
Validation sub-namespace	`validate`	`validation`
LAMMPS input script	`lammpsScript`	`script`
LAMMPS structure file	`lammpsData`	`structure`
Skip validation flag	not available	`run_validation`
Trajectory retrieval	always on	`retrieve_trajectory` (default `False`)

The atomic number (`atomicNumber`) is a *required* training input in both versions. It is not

hard-coded in the plugin: the user must supply the correct value for each chemical element (e.g. `Int(13)` for aluminium, `Int(5)` for boron).

To submit the workflow once the `inputs` dictionary is built:

```
from aiida.engine import run_get_node
from aiida.plugins import WorkflowFactory

MakeNNP = WorkflowFactory("n2p2.make_potential")
result, node = run_get_node(MakeNNP, **inputs)
print(f"Submitted MakeNNPWorkchain with PK: {node.pk}")
```

1.3 CalcJobs and parsers

Three `CalcJob` plugins mirror the n2p2 executables:

Table 3: Calculation and parser entry points.

Entry point	CalcJob	Executable
`n2p2.scale`	`nnpScaling`	`nnp-scaling`
`n2p2.train`	`nnpTraining`	`nnp-train`
`n2p2.predict`	`nnpPredict`	`nnp-predict`

CalcJobs were hardened for AiiDA compliance:

- explicit `code` input in `define()`;
- default scheduler resources and parser names;
- structured exit codes for missing output files;
- removal of debug `print()` statements from parsers;
- clearer error messages in scaling and predict parsers.

The training parser selects the best epoch from `learning-curve.out` (lowest test-set RMSE) and stores the corresponding weight file as `weights`. It also stores `last_weights` from the final completed epoch, which serves as the restart checkpoint for the next training session.

Remote output recovery (v0.4.0). On some HPC schedulers (notably OAR on Dahu), the default retrieve step can copy incomplete output files before the executable finishes. The scale and train parsers therefore poll the remote folder when needed: `scaling.data` and `learning-curve.out` are read only after the file size stabilises; training may prefer remote weight files when the learning curve lags behind the last written checkpoint. This allows automatic restart to proceed even when a CalcJob exits with a non-zero scheduler status but parseable partial outputs exist.

1.4 Structured inputs and restart (v0.3.0)

1.4.1 Data types

Two AiiDA data types wrap n2p2 input files with provenance-friendly metadata:

`N2p2Parameters` parses `input.nn` into an editable keyword dictionary. Keywords can be active, commented out, or flag-style (e.g. `#use_old_weights_short`). The method `prepare_for_restart()` enables `use_old_weights_short` and optionally extends `epochs`:

Table 4: Structured data entry points (v0.3.0).

Entry point	Class	Purpose
`n2p2.dataset`	`N2p2Dataset`	Training set (`input.data`)
`n2p2.parameters`	`N2p2Parameters`	Network config (`input.nn`)

```
from aiida.plugins import DataFactory

N2p2Parameters = DataFactory("n2p2.parameters")
params = N2p2Parameters.from_file("input.nn")
params.set("epochs", "20")
params.prepare_for_restart(additional_epochs=20)
input_nn = params.get_singlefile()
```

`N2p2PrepareInputsWorkChain` renders `input.data` and `input.nn` from these nodes (or from legacy `SinglefileData`), applying optional keyword overrides and restart preparation when `is_restart = Bool(True)`.

1.4.2 Two restart modes compared

Training can be continued in two ways. Both use the same CalcJob restart engine (`use_old_weights_short`, `weights.ZZZ.data`, merged learning curves), but differ in orchestration. Figure 2 contrasts the two modes in vertical form.

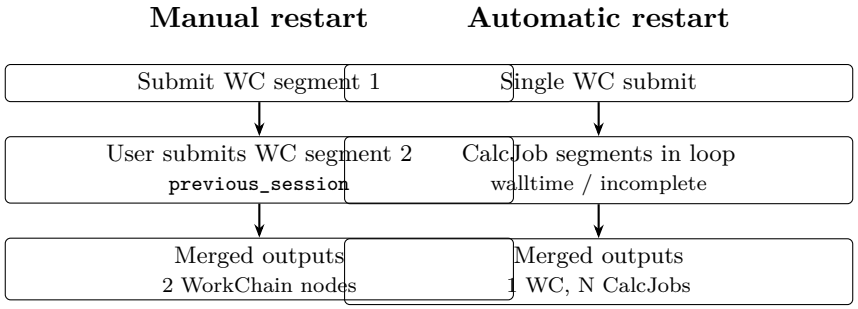


Figure 2: Manual vs. automatic training restart (compact two-column layout).

Table 5: Manual vs. automatic training restart in `N2p2TrainWorkChain`.

Aspect	Manual (v0.3)	Automatic (v0.4)
Inputs	`is_restart`, `previous_session`	`auto_restart`, `max_auto_restarts`
Who triggers next segment	User (new `submit`)	WorkChain (`while_` loop)
Typical use	Deliberate multi-step training	HPC walltime limits
Prior segment status	Usually `finished_ok`	May be failed (e.g. exit 143)
Parseable outputs required	Yes	Yes (`learning-curve.out`, `last_weights`)
AiiDA provenance	N WorkChain nodes	1 WorkChain, N CalcJob children
Extend `epochs`	`additional_epochs` (remaining in segment)	Fixed target in `input.nn`
Example script	`restart_train_demo.py`	`auto_restart_train_demo.py`

Manual restart (inter-workflow). The user submits one WorkChain per session and links them with `previous_session`. Each session is normally expected to finish successfully. This is appropriate when you deliberately split training (e.g. 40 + 40 epochs toward a target of 80) or when you want full control between steps. Each manual session is a separate `N2p2TrainWorkChain` submission.

Automatic restart (intra-workflow). The user submits *once*. If a CalcJob stops before `epochs` is reached in `input.nn` (e.g. scheduler walltime) but the parser recovered `learning-curve.out` and checkpoint weights, the WorkChain submits another CalcJob automatically until the target is met or `max_auto_restarts` is exceeded. This is appropriate on HPC clusters with fixed job time limits.

Output files and overwrite safety. Each CalcJob segment runs in a *new* remote folder (new AiiDA UUID). Segments do not overwrite each other's cluster files. Intermediate segments remain as child calculations in the provenance graph. The WorkChain output `learning_curve_merged` combines all segments; `weights` and `last_weights` refer to the final segment.

1.4.3 Manual restart API

`N2p2TrainWorkChain` accepts optional restart inputs:

- `is_restart` (`Bool`): restart from a previous session;
- `previous_session` (`WorkChainNode`, `non_db=True`): prior `N2p2TrainWorkChain` whose `last_weights` are reused;
- `parameters` (`N2p2Parameters`): template used to regenerate `input.nn` with restart keywords;
- `additional_epochs` (`Int`): number of *remaining* epochs written into the restart segment's `input.nn` (not added to the cumulative total in the file).

On manual restart the WorkChain sets `epochs` in the regenerated `input.nn` to `additional_epochs` after enabling `use_old_weights_short`. The global target for completion checks comes from `parameters` (e.g. `epochs=80` while the first segment used 40). Automatic restart uses the same "remaining epochs" convention internally.

On restart, the work chain copies `previous_session.outputs.last_weights` into the CalcJob sandbox as `weights.ZZZ.data`, sets `run_label` to the previous value plus one, and merges learning-curve segments into `learning_curve_merged` and `learning_curve_plot_data` (one colour per run, global epoch axis).

Example (two deliberate sessions, 40 + 40 epochs toward target 80):

```
from aiida.engine import run_get_node
from aiida.orm import Bool, Int
from aiida.plugins import WorkflowFactory

ScaleWC = WorkflowFactory("n2p2.scale")
TrainWC = WorkflowFactory("n2p2.train")

# Use examples/2.HPC/input.data and input.nn (Al dataset)
params.set("epochs", "40")
_, scaling = run_get_node(ScaleWC, ...)
_, run1 = run_get_node(TrainWC, ..., parameters=params)

params_restart = params.clone()
```

```
params_restart.set("epochs", "80")  # global target for WorkChain
_, run2 = run_get_node(
    TrainWC,
    ...,
    is_restart=Bool(True),
    previous_session=run1,
    parameters=params_restart,
    additional_epochs=Int(40),  # remaining epochs in segment 2
)
merged = run2.outputs.learning_curve_plot_data.get_dict()
assert merged["n_runs"] == 2
```

1.4.4 Automatic restart API (v0.4.0)

Additional inputs:

- `auto_restart` (`Bool`): loop until target `epochs`;
- `max_auto_restarts` (`Int`): safety limit (default 10).

Set `max_wallclock_seconds` in CalcJob metadata so each segment is stopped by the scheduler before the target is reached. The train parser records `target_epochs`, `training_completed`, and `epochs_remaining` in `training_summary`.

```
from aiida.engine import run_get_node
from aiida.orm import Bool, Int
from aiida.plugins import WorkflowFactory

TrainWC = WorkflowFactory("n2p2.train")

_, node = run_get_node(
    TrainWC,
    ...,
    parameters=params,
    auto_restart=Bool(True),
    max_auto_restarts=Int(10),
    metadata=Dict(dict={
        "options": {
            "resources": {"num_machines": 1},
            "max_wallclock_seconds": 7200,
        }
    }),
)
assert node.outputs.training_summary.get_dict()["training_completed"]
assert node.outputs.learning_curve_plot_data.get_dict()["n_runs"] >= 2
```

Walltime is enforced by the cluster scheduler; on a plain local AiiDA computer the job may complete in one segment and automatic restart will not trigger. Use `examples/3.Restart/auto_restart_dahu.py` on Dahu/OAR (default `max_wallclock_seconds=360`, target 300 epochs) or `auto_restart_train_demo.py` on other schedulers to verify end-to-end behaviour (`n_runs >= 2` in `learning_curve_plot_data`). Logic is also covered by unit tests without re-running n2p2.

Template leftovers from the original plugin scaffold (`helpers.py`, `cli.py`, unused `DiffParameters` data type) were removed.

1.5 Memory-conscious retrieval defaults

Two design choices reduce permanent storage in the AiiDA repository and avoid unnecessary data transfer on HPC systems.

Training weight files. n2p2 writes one `weights.ZZZ.EEEEEE.out` file per training epoch. Only the file corresponding to the best epoch is needed as a workflow output. Accordingly, in `nnpTraining`:

- `learning-curve.out` is listed in `retrieve_list` (stored permanently);
- `weights.*.out` is listed in `retrieve_temporary_list` (discarded after parsing).

The train parser reads the best weight file from the temporary retrieved folder and stores only that single file in the provenance graph. For a typical aluminium run with ∼200 epochs, this avoids archiving ∼200 redundant weight files (∼1.2 MB in the reference case, scaling to hundreds of megabytes for long training runs).

LAMMPS trajectories. By default, LAMMPS validation does *not* retrieve trajectory files (`*.lammpstrj`), which can grow to gigabytes for long molecular-dynamics runs. Retrieval is opt-in via `validation.retrieve_trajectory = Bool(True)`. Examples that post-process trajectories (e.g. radial distribution function analysis or structure visualisation) set this flag to `True`; production validation runs that only check exit status can leave it at the default `False`.

1.6 Regression tests

A fast regression test suite (48 tests, ∼6 s) validates parser logic, structured inputs, restart wiring, and learning-curve merging against golden outputs from a successful aluminium run (`MakeNNPWorkchain<949>`) without re-executing n2p2 or LAMMPS. Fixtures are stored under `tests/fixtures/al/` with a `REFERENCE.json` manifest recording:

- MD5 checksum of `scaling.data`;
- best training epoch (182), last epoch (200), and test-set RMSE;
- MD5 checksum of the selected best and last weights files.

Tests cover scale and train parser output, learning-curve epoch selection, plugin entry-point registration, work-chain port specifications, `input.nn` parsing, `N2p2Parameters` restart preparation, and CalcJob/WorkChain restart wiring. They are run with:

```
PYTHONPATH=. pytest -v
```

1.7 Updated examples

The following examples reflect the v0.4.0 API:

- `examples/1.Al/AiidA-n2p2_demo.ipynb` — local Al demonstration notebook (with `retrieve_trajector = Bool(True)` for trajectory visualisation);
- `examples/2.HPC/wkchain_Al.py` — full pipeline on HPC or localhost (`N2P2_COMPUTER`, code labels via environment variables);
- `examples/1.Boron/aiida-n2p2_demo.py` — boron example with `Int(5)` and `retrieve_trajectory = Bool(False)`;
- `examples/3.Restart/restart_train_demo.py` — manual restart: two WorkChain submits (default 40 + 40 epochs; Al inputs from `examples/2.HPC/`);
- `examples/3.Restart/auto_restart_dahu.py` — automatic restart on Dahu/OAR (`pr-diamond`, configurable walltime per segment);

- `examples/3.Restart/auto_restart_train_demo.py` — automatic restart smoke test (any scheduler; localhost walltime may not trigger multiple segments).

Restart examples require the full Al `input.data` from `examples/2.HPC/`; the minimal files under `examples/3.Restart/` are for unit tests only.